

VIETNAM NATIONAL UNIVERSITY – HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



PRINCIPLES OF DATABASE MANAGEMENT
IT079IU

Topic: Blood Donation

By Group 04 – Member List

NUMBER	STUDENT NAME	STUDENT ID	ROLE
1	Phạm Nguyễn Minh Mẫn	ITITDK23034	Leader
2	Nguyễn Đức Duy	ITITIU21008	Member
3	Đỗ Anh Quân	ITCSIU23057	Member
4	Võ Hoàng Cát Triệu	ITITDK23011	Member
5	Nguyễn Tấn Hiếu	ITCSIU22045	Member
6	Huỳnh Đạt Minh Tâm	ITITIU21305	Member
7	Đinh Quốc Bình	ITDSIU24008	Member
8	Nguyễn Anh Tuyên	ITITWE23030	Member
9	Nguyễn Minh Cát Tường	ITCSIU22295	Member
10	Hồ Gia Huy	ITDSIU23007	Member
11	Phạm Nguyễn Công Danh	ITDSIU23035	Member

Instructor: Assoc. Prof. Nguyen Thi Thuy Loan

TABLE OF CONTENTS

LIST OF TABLES.....	2
ABSTRACT.....	4
CHAPTER 1: INTRODUCTION.....	5
1.1. System Overview.....	5
1.2. Goal.....	5
1.3. The techniques & tools used.....	5
CHAPTER 2: TASK TIMELINE & DIVISION	7
2.1. Project timeline & task division.....	7
2.2. Contribution.....	9
CHAPTER 3: METHODOLOGY.....	10
3.1 Entity relationship diagram (ERD).....	10
3.1.1 ERD Diagram sample.....	10
3.1.2 Entity and attribute descriptions.....	10
3.1.3 Relationship descriptions.....	13
3.2 Schema diagram.....	14
3.3 Table Creation and Normalization (Ensuring 3NF).....	15
3.3.1 Table Creation.....	15
3.3.2 Normalization Analysis.....	18
3.3.3 Summary.....	19
3.4 Database Data Insertion.....	19
3.4.1 Source of Data Insertion.....	19
3.4.2 Data Insertion.....	19
3.5. Database Queries: Relational Algebra, Tree, and SQL Queries.....	30
3.6 Blood Donation Website.....	35
3.6.1 Website Structure.....	35
3.6.2 Demo website.....	37
CHAPTER 4: IMPLEMENTATION & DISCUSSION.....	44
4.1 Implementation Process.....	44
4.2 Results & Analysis.....	47
4.3 Challenges & Solutions.....	48
CHAPTER 5: CONCLUSION & RECOMMENDATION.....	49

5.1 Summary of Key Findings.....	49
5.2 Reflection on Objectives.....	49
5.3 Implications and Significance.....	50
5.4 Future Work.....	50
5.5 Conclusion.....	50
REFERENCES.....	51
LIST OF TABLES.....	Page
Table 2.1 Sprint planning and task division for individuals and teams	7
Table 2.2 Individual responsibility and contribution.....	9
LIST OF FIGURES.....	Page
Figure 3.1.1: ERD.....	10
Figure 3.2: Database Schema	14
Figure 3.6.2.1: Homepage Website.....	38
Figure 3.6.2.2: Login Account.....	38
Figure 3.6.2.3: Blood Bank Management.....	39
Figure 3.6.2.4: Blood Inventory.....	39
Figure 3.6.2.5: Blood Request.....	40
Figure 3.6.2.6: Donation Event.....	40
Figure 3.6.2.7: Doctor Dashboard.....	41
Figure 3.6.2.8: Patient Dashboard.....	41
Figure 3.6.2.9: Donor.....	41
Figure 3.6.2.10: Blood Bank List.....	42
Figure 3.6.2.10: General Dashboard.....	42

ABSTRACT

The Database Management course project involves developing a Blood Donation Management System using Java and MySQL as the core back-end technologies, with GitHub employed for version control and progress tracking among team members. The system includes key features such as donor account creation, doctor management, blood bank records, and inventory tracking. It emphasizes storing and managing donor information, doctor assignments, blood bank data, and blood inventory details to ensure quick access to donation history, blood requests, and availability status.

The database design is carefully planned to maintain efficiency, consistency, and scalability, with well-defined data entities, relationships, attributes, and constraints. A user-friendly interface is incorporated to support intuitive navigation, enabling hospitals, blood banks, and doctors to access system features conveniently and improve the overall experience.

The project follows an agile methodology (preparation, implementation, and testing) with regular sprints and feedback sessions to ensure that the application meets course requirements and achieves stable performance. The project team, consisting of developers and testers, collaborates closely to deliver a high-quality product ready for real-world implementation.

Overall, this project provides practical experience in applying database management principles to a healthcare-related system, highlighting the importance of efficient data management in ensuring transparency, reliability, and availability of blood resources.

CHAPTER 1: INTRODUCTION

1.1 System Overview

Blood donation is a crucial activity that saves millions of lives every year. However, many blood banks and hospitals still face challenges in managing donor information, tracking blood inventory, and handling requests efficiently. Traditional manual record-keeping often results in errors, data duplication, and delays in responding to urgent needs.

The Blood Bank Database Management System is designed to address these challenges by providing a structured and reliable database solution. The system manages key entities, including donors, doctors, blood banks, blood inventory, donations, and requests. Donors can register and update their personal and medical details, while doctors supervise donation activities and verify donor eligibility. Blood inventory is systematically tracked by type and availability to ensure that there are no shortages or wastage. Requests from hospitals or individuals can be processed quickly and accurately to match available blood units with demand.

The system will be developed using Entity Relationship Diagrams (ERD) and Database Schema Design. ERDs provide a clear visualization of the entities and their relationships, while schemas define tables, attributes, and constraints. Together, they ensure accurate modeling, efficient SQL queries, and reliable database management.

1.2 Goal

The goal of this project is to design and implement a secure, efficient, and user-friendly blood bank database system that improves the management of blood donation activities. The main objectives include:

- To organize donor and doctor information systematically.
- To record and track donations and blood requests accurately.
- To maintain updated records of blood inventory categorized by blood type and quantity.
- To ensure real-time access to critical data for decision-making, especially during emergencies.
- To reduce human errors, eliminate redundant records, and streamline the workflow of blood banks.
- To provide a strong foundation for future system expansions, such as integrating a web or mobile application for end-users.

Additionally, this project allows students to gain hands-on experience in database design, schema modeling, normalization, query optimization, and SQL implementation, thereby enhancing both technical and problem-solving skills.

1.3 Techniques & Tools Used

- **GitHub:** Used for version control, progress tracking, and collaborative work among team members.
- **Database Server:** MySQL is chosen as the database server due to its open-source availability, strong ACID compliance, and capability to handle complex relational data.

- **Database Management System (DBMS):** MySQL provides optimized indexing and high-performance queries, making it suitable for managing donor and blood inventory records.
- **Entity Relationship Diagram (ERD):** Designed using tools namely Draw.io and ERDplus to visualize entities and relationships clearly.
- **Database Schema Design:** Derived from the ERD to define tables, attributes, primary and foreign keys, and constraints.
- **Database Interactions:** SQL queries (CREATE, INSERT, UPDATE, DELETE, SELECT, JOIN) will be written to perform operations on the database efficiently.
Integrated Development Environment (IDE): MySQL Workbench will be used for designing ERDs, implementing schemas, testing queries, and visualizing results.
- **Documentation Tools:** Microsoft Word or Google Docs will be used for preparing reports, while diagrams are included for better explanation and understanding.
- **Visual Studio Code (VS Code):** The primary IDE for coding and debugging.
- **Spring Boot (Java):** To develop the backend API and manage business logic.
- **JDBC (Java Database Connectivity):** For seamless interaction with the MySQL database.

CHAPTER 2: TASK TIMELINE & DIVISION

2.1 Task timeline & division

Member's Name	Tasks	Deadline
Proposal Reporting Period (chapter 1)		
All members	Choose a topic and discuss.	9/9 - 12/9/2025
Đỗ Anh Quân	Write an introduction for the proposal and edit the reporting file (chapter 1).	13/9 - 16/9/2025

Phạm Nguyễn Minh Mẫn	Write a timeline for the proposal and edit the reporting file (chapter 2).	13/9 - 16/9/2025
Midterm Reporting Period (chapter 2 and chapter 3)		
All members	Configure, educate, and establish a working environment on Github and SQL.	17/09 - 24/9/2025
Hồ Gia Huy	Design the Entity-Relationship Diagram (ERD) of Donation Event & Request entities.	25/9 - 12/10/2025
Nguyễn Minh Cát Tường	Design the Entity-Relationship Diagram (ERD) of Blood Bank & User entities.	25/9 - 12/10/2025
Phạm Nguyễn Công Danh	Design the Entity-Relationship Diagram (ERD) of (Donor / Patient) & Doctor entities.	25/9 - 12/10/2025
Đinh Quốc Bình	Design the Entity-Relationship Diagram (ERD) of Blood Inventory entities.	25/9 - 12/10/2025
Nguyễn Tấn Hiếu	Determine the whole relationship of the Entity-Relationship Diagram (ERD).	25/9 - 17/10/2025
Huỳnh Đạt Minh Tâm	Design the schema and create a table of Donor and Patient entities.	13/10 - 20/10/2025
Võ Hoàng Cát Triệu	Design the schema and create a table of Blood Bank & User (Account) entities.	13/10 - 20/10/2025
Nguyễn Anh Tuyên	Design the schema and create a table of Request and Donation event entities.	13/10 - 20/10/2025
Phạm Nguyễn Minh Mẫn	Design the schema and create an integrated database and a table of Blood Inventory & Doctor entities.	13/10 - 20/10/2025
Nguyễn Đức Duy	Create relationships and ensure Third Normal Form (3NF) normalization in table and schema.(1NF and 2NF).	13/10 - 20/10/2025
Đỗ Anh Quân	Create relationships and ensure Third Normal Form (3NF) normalization in table and schema.(3NF and Summary).	11/10 - 20/10/2025
Phạm Nguyễn Minh Mẫn	Manage the midterm report for the midterm and edit the reporting file (chapter 2, 3)	11/10 - 20/10/2025
Final Reporting Period (chapter 3, 4 and chapter 5)		
Võ Hoàng Cát Triệu	Report chapter 3	5/11 - 21/11/2025
Phạm Nguyễn Minh Mẫn	Full Stack development	5/11 - 21/11/2025

Đỗ Anh Quân	Report chapter 4 and 5	5/11 - 21/11/2025
Nguyễn Đức Duy	presentation and Q&A for Entity Relationship Diagram	5/11 - 22/11/2025
Nguyễn Tấn Hiếu	presentation and Q&A for Entity Relationship Diagram	5/11 - 22/11/2025
Nguyễn Anh Tuyên	presentation and Q&A for Schema	5/11 - 22/11/2025
Đinh Quốc Bình	presentation and Q&A for Schema	5/11 - 22/11/2025
Hồ Gia Huy	presentation and Q&A for Database and Query	5/11 - 22/11/2025
Phạm Nguyễn Công Danh	presentation and Q&A for Database and Query	5/11 - 22/11/2025
Huỳnh Đạt Minh Tâm	presentation and Q&A for Demo Website	5/11 - 22/11/2025
Nguyễn Minh Cát Tường	presentation and Q&A for Demo Website	5/11 - 22/11/2025
Phạm Nguyễn Minh Mẫn	Integrate the slide presentation	5/11 - 22/11/2025
All member	Meeting for discussion and rehearsal. Check errors and unify information. Remind yourself of the knowledge before presenting.	5/11 - 23/11/2025

2.2 Contribution

Member's Name	Member's ID	Evaluation					Contribution
		Midterm & Proposal (40 %)		Final Proposal (60 %)			
Phạm Nguyễn Minh Mẫn	ITITDK23034	20%	20%	20%	20%	20%	100%
Nguyễn Đức Duy	ITITIU21008	20%	20%	20%	20%	20%	100%
Đỗ Anh Quân	ITCSIU23057	20%	20%	20%	20%	20%	100%
Võ Hoàng Cát Triệu	ITITDK23011	20%	5%	15%	10%	20%	60%
Nguyễn Tấn Hiếu	ITCSIU22045	20%	20%	20%	20%	20%	100%
Huỳnh Đạt Minh Tâm	ITITIU21305	0%	0%	0%	0%	0%	0%
Đinh Quốc Bình	ITDSIU24008	20%	20%	20%	20%	20%	100%
Nguyễn Anh Tuyên	ITITWE23030	5%	5%	20%	5%	20%	55%
Nguyễn Minh Cát Tường	ITCSIU22295	20%	20%	20%	20%	20%	100%
Hồ Gia Huy	ITDSIU23007	20%	20%	20%	20%	20%	100%
Phạm Nguyễn Công Danh	ITDSIU23035	20%	20%	20%	20%	20%	100%

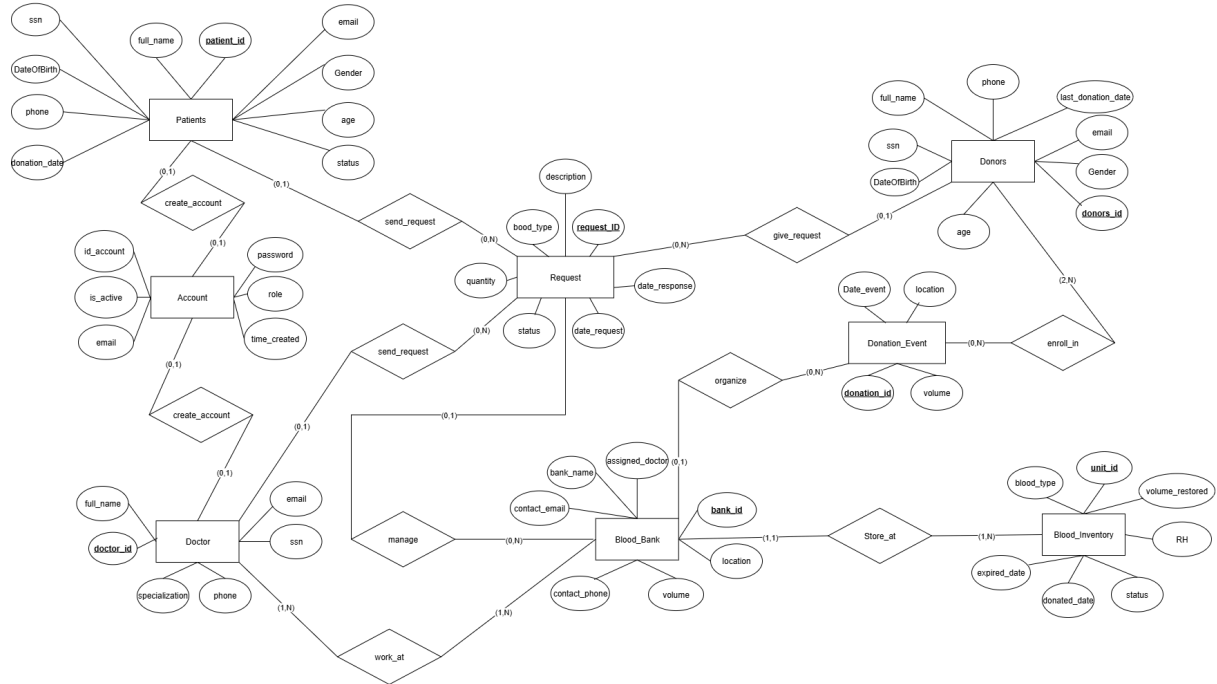
Attached file:  PDM contribution

CHAPTER 3: METHODOLOGY

3.1 Entity relationship diagram (ERD):

3.1.1 ERD Diagram sample:

Figure 3.1.1 ERD for Blood Donation Management System (Cardinality Ratio)



3.1.2 Entities and Attributes Descriptions:

→ Account:

- ◆ **Description:** represents a user login or credentials record within the system, necessary for authentication and access control.
- ◆ **Key Attribute:** - **Account ID** (Primary Key). The attribute name suggests it is the unique identifier for all types of accounts.
- ◆ **Basic Attributes:**
 - **email:** the unique email address associated with the account, often used for login and communication.
 - **password:** the encrypted or hashed password for account authentication.
 - **role:** the access level or function of the user associated with this account (e.g., Patient, Doctor, Bank Staff, Admin).
 - **is_active:** a flag indicating whether the account is currently enabled or disabled.
 - **time_created :** the date and time the account was initially registered in the system.

→ **Patients:**

- ◆ **Description:** Represent the record of an individual receiving medical treatment or requiring blood/services from the blood bank.
- ◆ **Key Attribute:** - **patient_id** (Primary key)
- ◆ **Basic attributes:**
 - **full_name:** the patient's full name.
 - **DateOfBirth:** the patient's date of birth, essential for determining age and planning age-specific medical requirements.
 - **Gender:** the patient's record of gender (male/female).
 - **ssn:** the patient's Social Security Number (or national identity number).
 - **phone:** the patient's phone number, used for contact.
 - **email:** the primary email address of the patient, used for contact and sending notifications.
 - **status:** the current condition of the patient.
 - **age:** the patient's current age.
 - **donation_date:** the date the patient last received blood.

→ **Doctor:**

- ◆ **Description:** Represent a doctor who works at the blood bank.
- ◆ **Key Attribute:** **doctor_id** (Primary key)
- ◆ **Basic Attributes:**
 - **full_name:** the full name of the doctor
 - **phone:** the donor's phone number, used for contact.
 - **specialization:** the doctor's area of medical expertise, used for task allocation.
 - **ssn:** the doctor's Social Security Number (or national identity number).
 - **email:** the primary email address of the doctor.

→ **Donors:**

- ◆ **Description:** Represents a record of an individual who is eligible and willing to donate blood. This entity is the source of all blood donations.
- ◆ **Key Attribute:** **donor_id** (Primary key)
- ◆ **Basic Attributes:**
 - **last_donation_date:** the date of the donor's most recent successful blood donation.
 - **email:** the primary email address of the donor, used for contact and sending notifications (e.g., test results).
 - **Gender:** the donor's record of gender (male/female).
 - **age:** the donor's current age
 - **phone:** the donor's phone number, used for contact.
 - **ssn:** the donor's Social Security Number (or national identity number).
 - **DateOfBirth:** the donor's date of birth, used to verify age eligibility and calculate their current age.
 - **full_name:** the full name for the donor.

→ **Request:**

- ◆ **Description:** Represents a formal request for blood made by patients, indicating the required blood type, quantity, and status of the request. It is used by doctors and the blood bank to process and fulfill blood demands efficiently.
- ◆ **Key attribute:** **request_id** – Request (Primary key)
- ◆ **Basic attribute:**
 - **blood_type:** type of blood requested (e.g., A+, A-, B+, B-, AB+, AB-, O+, O-).
 - **quantity:** amount of blood required (in milliliters or units).
 - **status:** current status of the request (e.g., Pending, Approved, Fulfilled, Rejected).
 - **date_request:** date the request was made.
 - **date_response:** date and time the request was last updated or fulfilled.
 - **description:** particular details about the request.

→ **Donation_Event:**

- ◆ **Description:** Represents the act of donating blood by a donor, recorded and managed by the blood bank. Each donation may fulfill a specific blood request and contributes to maintaining the hospital's blood inventory
- ◆ **Key attribute:** **donation_id** (Primary key)
- ◆ **Basic attributes:**
 - **volume:** the amount of blood donated (in milliliters).
 - **date:** the date the donation took place.
 - **location:** the physical location (e.g., hospital, blood bank, donation event).

→ **Blood_Bank:**

- ◆ **Description:** represents a facility or institution where blood is collected, stored, and managed.
- ◆ **Key Attribute:** **bank_id** - Blood Bank ID (Primary Key). It is underlined in the diagram, indicating it is the key identifier.
- ◆ **Basic Attributes:**
 - **bank_name:** the official name of the blood bank.
 - **contact_email:** the primary email address for the bank.
 - **contact_phone:** the primary phone number for the bank.
 - **location:** the physical address or general location of the bank.
 - **volume:** likely represents the current total volume or capacity of blood stock (inventory) held by the bank.
 - **assigned_doctor:** the doctor or medical professional currently assigned to oversee the operations of the bank.

→ **Blood_Inventory:**

- ◆ **Description:** helps track blood units effectively to ensure proper usage before expiration.
- ◆ **Key Attribute:** **unit_id** - *Primary Key (PK)*. Uniquely identifies each blood unit in the inventory.
- ◆ **Basic Attributes:**

- **blood_type**: Indicates the blood group (A, B, AB, or O).
- **volume**: The amount of blood contained in the unit (e.g., in milliliters).
- **RH**: Rh factor of the blood (positive or negative).
- **expired_date**: The date when the blood unit becomes unusable.
- **donated_date**: The date when the blood unit was collected or donated.
- **status**: Represents the availability condition (Available, Reserved, or Finished).

3.1.3 Relationship descriptions :

→ Patients – Account

- ◆ Relationship: create_account
- ◆ Type: (0,1) ↔ (0,1)
- ◆ Meaning : Each patient may create one account, and each account belongs to at most one patient.
→ A patient can log in and manage their donation requests via this account.

→ Doctor – Account

- ◆ Relationship: created_account
- ◆ Type: (0,1) ↔ (0,1)
- ◆ Meaning : Each doctor can have one login account, and each account is linked to one doctor.
→ Used for hospital staff or blood bank staff authentication.

→ Patients – Request

- ◆ Relationship: send_request
- ◆ Type: (0,1) ↔ (0,N)
- ◆ Meaning : A patient can send multiple blood requests, but a request must come from one patient.
→ Represents the act of a patient requesting blood.

→ Donors – Request

- ◆ Relationship: give_request
- ◆ Type: (0,1) ↔ (0,N)
- ◆ Meaning : A donor can respond to multiple requests, while each request may be fulfilled by one donor.
→ Tracks which donor responded to which patient requests.

→ Doctor – Request

- ◆ Relationship: manage_request
- ◆ Type: (0,1) ↔ (0,N)
- ◆ Meaning : A doctor can manage multiple requests, but each request is managed by one doctor.
→ Represents the review or approval of blood requests by medical staff.

→ Doctor – Blood Bank

- ◆ Relationship: work_at
- ◆ Type: (N, N)
- ◆ Meaning : Each doctor works at many blood banks, and one blood bank employs multiple doctors.
→ Connect the medical staff to the facility they belong to.

→ Blood Bank – Donation Event

- ◆ Relationship: organize
- ◆ Type: (1,N)
- ◆ Meaning : A blood bank can organize multiple donation events, and each event is organized by one blood bank.
→ Tracks which bank is responsible for which donation campaign.

→ **Donors – Donation_Event**
 ◆ Relationship: enroll_in
 ◆ Type: (N, N)

- ◆ Relationship: enroll_in
- ◆ Type: (N, N)
- ◆ Meaning : Each donor can enroll in multiple donation events, and each event can have multiple donors.
→ Represents donor participation in donation drives.

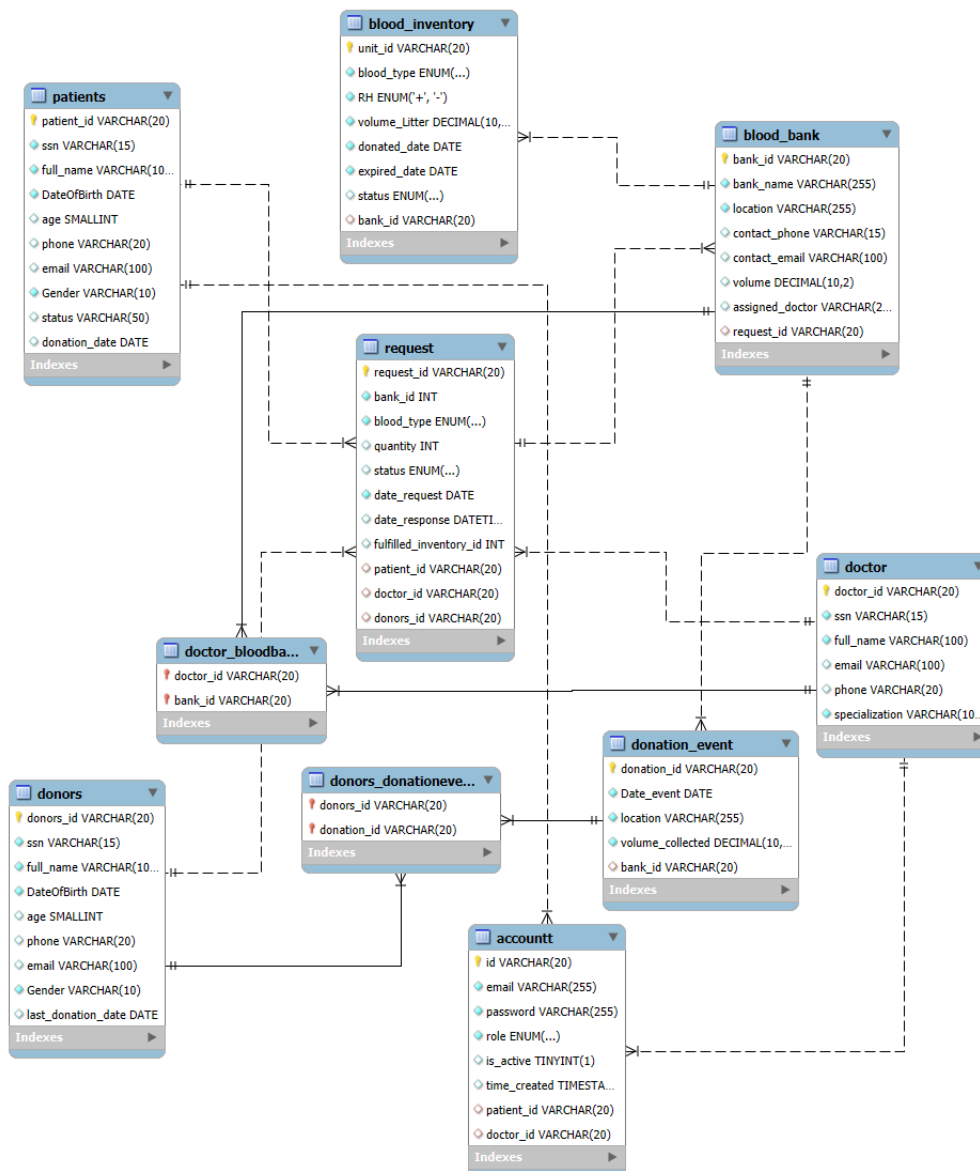
→ **Blood Bank – Blood Inventory**

- ◆ Relationship: store_at
- ◆ Type: (1, N)
- ◆ Meaning : Each blood inventory is stored at various blood banks, and a blood bank just has one blood inventory.
→ Represents storage and tracking of blood units.

→ Request – Blood_Bank

- ◆ Relationship: manage
- ◆ Type: $(0,1) \leftrightarrow (0,N)$
- ◆ Meaning : Each request is just managed by one blood bank , while one blood bank could manage various requests → A patient can log in and manage their donation requests via this account.

3.2 Schema diagram:



3.3 Table Creation and Normalization (Ensuring 3NF):

In order to make sure the design satisfies at least the Third Normal Form (3NF), this section examines the normalization level attained and describes the development of tables within the Blood Donation database.

3.3.1 Table Creation

The logical database design, represented by the relational model schemas, has been implemented physically using SQL CREATE TABLE statements. These statements define the structure for each table required by the system: Patients, Account, Request, Doctor, Blood_Bank, Donor, Donation_Event, Blood_Inventory . Each CREATE TABLE statement specifies.

- ◆ **Table name:** the identifier for the relation (ex: Donor)
- ◆ **Attribute (column):** the data elements stored within the table (ex: ssn)
- ◆ **Data types:** each attribute's permitted data type (INT, VARCHAR, ENUM, DATE, BOOLEAN, TEXT, DECIMAL, TIMESTAMP, etc.)
- ◆ **Constraints:** Rules applied to the data.
- ◆ **Storage Engine:**

◆ **Attached file (Database):**

◆ **Data creation:**

```
CREATE DATABASE blood_donation;  
USE blood_donation;
```

◆ **Table creation:**

```
-- DOCTORS  
CREATE TABLE IF NOT EXISTS Doctor (  
  doctor_id    VARCHAR(20) PRIMARY KEY,  
  ssn          VARCHAR(15) UNIQUE NOT NULL,  
  full_name    VARCHAR(100) NOT NULL,  
  email        VARCHAR(100) UNIQUE,  
  phone        VARCHAR(20),  
  specialization VARCHAR(100) NOT NULL,  
  CONSTRAINT check_specialization_format CHECK (specialization IS NOT  
  NULL AND specialization <> "")  
);
```

```
-- PATIENTS  
CREATE TABLE IF NOT EXISTS Patients (  
  patient_id   VARCHAR(20) PRIMARY KEY,  
  ssn          VARCHAR(15) UNIQUE NOT NULL,  
  full_name    VARCHAR(100) NOT NULL,  
  DateOfBirth  DATE NOT NULL,  
  age          SMALLINT,
```

```

phone      VARCHAR(20),
email      VARCHAR(100) UNIQUE,
Gender     VARCHAR(10) NOT NULL,
status     VARCHAR(50) DEFAULT 'New',
donation_date DATE,
CONSTRAINT check_patient_age CHECK (age > 0),
CONSTRAINT check_patient_gender CHECK (Gender IN ('Male', 'Female',
'Other'))
);

```

```

-- Donors
CREATE TABLE IF NOT EXISTS Donors (
donors_id   VARCHAR(20) PRIMARY KEY,
ssn         VARCHAR(15) UNIQUE NOT NULL,
full_name   VARCHAR(100) NOT NULL,
DateOfBirth DATE NOT NULL,
age         SMALLINT,
phone       VARCHAR(20),
email       VARCHAR(100) UNIQUE,
Gender      VARCHAR(10) NOT NULL,
last_donation_date DATE,
CONSTRAINT check_donor_age CHECK (age >= 18 AND age <= 100),
CONSTRAINT check_donor_gender CHECK (Gender IN ('Male', 'Female',
'Other'))
);

```

```

-- ACCOUNT
CREATE TABLE IF NOT EXISTS ACCOUNTT(
id      VARCHAR(20) PRIMARY KEY,
email   VARCHAR(255) UNIQUE NOT NULL,
password VARCHAR(255) NOT NULL,
role    ENUM ('Patient', 'Doctor') NOT NULL,
is_active BOOLEAN DEFAULT TRUE,
time_created TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
patient_id VARCHAR(20),
doctor_id  VARCHAR(20),
FOREIGN KEY (patient_id) REFERENCES Patients (patient_id),
FOREIGN KEY (doctor_id) REFERENCES Doctor (doctor_id)
);

```

```

-- BLOOD_BANK
CREATE TABLE IF NOT EXISTS BLOOD_BANK (
bank_id VARCHAR(20) PRIMARY KEY,
bank_name VARCHAR(255) NOT NULL,
location VARCHAR(255) NOT NULL,
contact_phone VARCHAR(15),
contact_email VARCHAR(100),
volume DECIMAL(10,2) DEFAULT 0,
assigned_doctor VARCHAR(20),

```



```

    request_id VARCHAR(20),
    FOREIGN KEY (request_id) REFERENCES Request (request_id)
);

```

-- DONATION EVENT

```

CREATE TABLE IF NOT EXISTS Donation_Event (
    donation_id VARCHAR(20) PRIMARY KEY,
    Date_event DATE NOT NULL,
    location VARCHAR(255) NOT NULL,
    volume_collected DECIMAL(10, 2) NOT NULL,
    bank_id VARCHAR(20),
    FOREIGN KEY (bank_id) REFERENCES BLOOD_BANK (bank_id)
) ENGINE=InnoDB;

```

-- BLOOD INVENTORY

```

CREATE TABLE IF NOT EXISTS Blood_Inventory(
    unit_id VARCHAR(20) PRIMARY KEY,
    blood_type ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,
    RH ENUM('+', '-') NOT NULL,
    volume_Litter DECIMAL(10,2) NOT NULL,
    donated_date DATE NOT NULL,
    expired_date DATE NOT NULL,
    status ENUM('available', 'used', 'expired') DEFAULT 'available',
    bank_id VARCHAR(20),
    CONSTRAINT chk_dates CHECK (expired_date > donated_date),
    FOREIGN KEY (bank_id) REFERENCES BLOOD_BANK(bank_id)
) ENGINE=InnoDB;

```

-- request table

```

CREATE TABLE IF NOT EXISTS Request (
    request_id VARCHAR(20) PRIMARY KEY,
    bank_id INT NOT NULL,
    blood_type ENUM('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-') NOT NULL,
    quantity INT CHECK (quantity > 0),
    status ENUM('Pending', 'Approved', 'Fulfilled', 'Rejected') DEFAULT 'Pending',
    date_request DATE NOT NULL,
    date_response DATETIME NULL,
    fulfilled_inventory_id INT NULL,
    patient_id VARCHAR(20),

```

-- Associative table

```

CREATE TABLE IF NOT EXISTS Donors_DonationEvent (
    donors_id VARCHAR(20) NOT NULL,
    donation_id VARCHAR(20) NOT NULL,
    PRIMARY KEY (donors_id, donation_id),
    FOREIGN KEY (donors_id) REFERENCES Donors(donors_id),
    FOREIGN KEY (donation_id) REFERENCES Donation_Event(donation_id)

```

);

```
CREATE TABLE IF NOT EXISTS Doctor_BloodBank (  
  doctor_id    VARCHAR(20) NOT NULL,  
  bank_id      VARCHAR(20) NOT NULL,  
  PRIMARY KEY (doctor_id, bank_id),  
  FOREIGN KEY (doctor_id) REFERENCES Doctor(doctor_id) ON DELETE  
  CASCADE,  
  FOREIGN KEY (bank_id) REFERENCES Blood_Bank(bank_id) ON DELETE  
  CASCADE  
);
```

3.3.2 Normalization Analysis

The practice of organizing a relational database according to a set of normal forms in order to minimize data redundancy and enhance data integrity is known as database normalization. The Blood Donation database is designed in accordance with the following normal forms:

Normal form	Description
1NF	Each column holds atomic values (no lists, no repeating groups)
2NF	All non-key attributes depend on the full primary key
3NF	No transitive dependency (non-key depends only on key)

1NF: The table must have a primary key, and all attributes (columns) must contain atomic (indivisible) values. There should be no repeating groups of columns. All defined tables (Users, Doctor, Patient, etc.) have a designated PRIMARY KEY. Each attribute is designed to hold a single value (e.g., Patient.name holds one name, Patient.phone holds one phone number). There are no repeating groups. Thus, the schema satisfies 1NF.

For example : All entities in the ERD — such as Patients, Donors, Doctor, Blood_Bank, and Request — contain atomic (indivisible) attributes. Each column holds a single value; for example, Patients has only one phone, one email, one gender per record. There are no repeating groups or multi-valued attributes. Each table has a clearly defined primary key (e.g., patient_id, donor_id, doctor_id, etc.), so the schema fully satisfies 1NF.

2NF: The table must be in 1NF, and all non-key attributes must be fully functionally dependent on the entire primary key. This form primarily addresses potential issues in tables with composite primary keys. The schema meets 1NF requirements. Furthermore, all primary keys in the defined tables are single-attribute keys. As there are no composite primary keys, there cannot be any partial dependencies (where a non-key attribute depends on only part of the primary key). Therefore, the schema automatically satisfies 2NF.

For example : Since all tables in the ERD use a single-attribute primary key, there are no partial dependencies (i.e., no non-key attribute depends on part of a composite key).

For instance, in the Request table, attributes such as quantity, status, and date_request depend entirely on request_id. Therefore, all non-key attributes are fully functionally dependent on the entire primary key, meaning the schema satisfies 2NF.

3NF: The Blood Donation Management System was normalized up to Third Normal Form (3NF) to ensure data consistency and eliminate redundancy. Each entity (Patients, Donors, Doctor, Blood Bank, etc.) has a unique primary key, and all non-key attributes depend only on the key.

3.3.3 Summary

The database schema, implemented via the provided SQL CREATE TABLE statements, has been designed following normalization principles up to the Third Normal Form (3NF). This structure minimizes data redundancy, avoids update/delete/insert anomalies, ensures data integrity through constraints and referential integrity, and provides a solid foundation for the application's data management needs..

3.4 Database Data Insertion

3.4.1 Source of Data Insertion:

For testing and demonstration purposes, the sample data for this project was manually generated by team members. The dataset represents typical scenarios within a blood donation management system, including multiple user roles (Doctors, Patients), blood banks, donors with varying eligibility statuses, donation histories, blood inventories across different types, and blood requests. Specifically, the dataset includes 5 Doctors, 6 Patients, 4 Blood Banks, and 10 Donors, along with their associated accounts, donation events, blood inventory entries, donor-event mappings, and blood requests. All data values are illustrative but structured to maintain referential integrity, providing a functional dataset for testing the database schema and evaluating application features.

3.4.2 Data Insertion: data.sql

A preview of each table's first few rows is shown below to show the data structure and offer example content. The project files or repository that are attached contain the full SQL script that contains the entire dataset (data.sql).

To meet foreign key constraints, the following SQL INSERT statements must be run in the designated order. Errors will occur during data insertion if this order is not followed. The primary keys, donor_id, doctor_id, donation_id, and request_id, are generic VARCHAR(20).

● Doctor Table:

```
INSERT INTO Doctor (doctor_id, ssn, full_name, email, phone, specialization) VALUES
('DOC001','012345678901','NguyễnMinhTuấn','nguyenvtuan.doc1@example.com','0391123001','Hematology'),('DOC002','0123
45678902','TrầnThịHồng','tranthihong.doc2@example.com','0391123002','TransfusionMedicine'),('DOC003','012345678903','Lê
VănHùng','levanhung.doc3@example.com','0391123003','InternalMedicine'),('DOC004','012345678904','PhạmThịLan','phamthil
an.doc4@example.com','0391123004','Pathology'),('DOC005','012345678905','HoàngVănSơn','hoangvanson.doc5@example.co
m','0391123005','EmergencyMedicine'),('DOC006','012345678906','VũThịMai','vuthimai.doc6@example.com','0391123006','Pe
diatrics'),('DOC007','012345678907','ĐặngQuốcAnh','dangquocanh.doc7@example.com','0391123007','Surgery'),('DOC008','0
```

```
12345678908','BùiThịNhung','buiithinhung.doc8@example.com','0391123008','Anesthesiology'),('DOC009','012345678909','Đỗ
VănNam','dovannam.doc9@example.com','0391123009','Cariology'),('DOC010','012345678910','NgôThịThanh','ngothithanh.do
c10@example.com','0391123010','Obstetrics & Gynecology');
```

● Patient Table:

```
INSERT INTO Patients (patient_id, ssn, full_name, DateOfBirth, age, phone, email, Gender, status,
donation_date) VALUES
('PAT001','10100010001','Nguyễn Văn
An','1985-03-12',40,'0371234001','nguyenvanan.pat1@example.com','Male','New',NULL),
('PAT002','10100010002','Trần Thị
Bích','1992-07-25',32,'0371234002','tranthibich.pat2@example.com','Female','New',NULL),
('PAT003','10100010003','Lê Minh
Châu','2001-11-04',23,'0371234003','leminhchau.pat3@example.com','Male','New',NULL),
('PAT004','10100010004','Phạm Thị
Dung','1978-02-18',47,'0371234004','phamthidung.pat4@example.com','Female','New',NULL),
('PAT005','10100010005','Hoàng Văn
Đức','1969-09-30',55,'0371234005','hoangvanduc.pat5@example.com','Male','New',NULL),
('PAT006','10100010006','Vũ Thị
Hạnh','1998-05-14',27,'0371234006','vuthihanh.pat6@example.com','Female','New',NULL),
('PAT007','10100010007','Đặng Văn
Khoa','1989-01-20',36,'0371234007','dangvankhoa.pat7@example.com','Male','New',NULL),
('PAT008','10100010008','Bùi Thị
Lệ','2005-10-03',19,'0371234008','buitile.pat8@example.com','Female','New',NULL),
('PAT009','10100010009','Đỗ Minh
Long','1995-12-11',29,'0371234009','diminhlong.pat9@example.com','Male','New',NULL),
('PAT010','10100010010','Ngô Thị
My','1982-06-01',42,'0371234010','ngothimy.pat10@example.com','Female','New',NULL),
('PAT011','10100010011','Phan Văn
Nam','1975-08-22',50,'0371234011','phanvannam.pat11@example.com','Male','New',NULL),
('PAT012','10100010012','Trịnh Thị
Oanh','1990-04-09',34,'0371234012','trinhthioanh.pat12@example.com','Female','New',NULL),
('PAT013','10100010013','Trương Minh
```

Phúc','1987-11-29',37,'0371234013','truongminhphuc.pat13@example.com','Male','New',NULL),
('PAT014','10100010014','Lý Thị
Quỳnh','1996-02-13',28,'0371234014','lythiquynh.pat14@example.com','Female','New',NULL),
('PAT015','10100010015','Võ Văn
Sang','2000-09-08',24,'0371234015','vovansang.pat15@example.com','Male','New',NULL),
('PAT016','10100010016','Phạm Thị
Tâm','1983-12-17',41,'0371234016','phamthitam.pat16@example.com','Female','New',NULL),
('PAT017','10100010017','Nguyễn Hữu
Tiền','1994-03-05',31,'0371234017','nguyenhuutien.pat17@example.com','Male','New',NULL),
('PAT018','10100010018','Trần Văn
Trung','1970-07-19',55,'0371234018','tranvantrung.pat18@example.com','Male','New',NULL),
('PAT019','10100010019','Lê Thị
Uyên','1988-10-21',36,'0371234019','lethiyen.pat19@example.com','Female','New',NULL),
('PAT020','10100010020','Phạm Quang
Vinh','1999-01-30',26,'0371234020','phamquangvinh.pat20@example.com','Male','New',NULL),
('PAT021','10100010021','Hoàng Thị
Xuyến','1986-05-27',39,'0371234021','hoangthixuyen.pat21@example.com','Female','New',NULL),
('PAT022','10100010022','Vũ Minh
Yên','2002-08-16',22,'0371234022','vuminhyen.pat22@example.com','Male','New',NULL),
('PAT023','10100010023','Đặng Thị
Ánh','1991-09-02',33,'0371234023','dangthianh.pat23@example.com','Female','New',NULL),
('PAT024','10100010024','Bùi Văn
Bình','1980-11-12',44,'0371234024','buivanbinh.pat24@example.com','Male','New',NULL),
('PAT025','10100010025','Đỗ Thị
Cầm','1973-04-04',52,'0371234025','dothicam.pat25@example.com','Female','New',NULL),
('PAT026','10100010026','Ngô Văn
Duy','1997-06-06',28,'0371234026','ngovanduy.pat26@example.com','Male','New',NULL),
('PAT027','10100010027','Phan Thị
Hương','1984-12-02',40,'0371234027','phanthihuong.pat27@example.com','Female','New',NULL),
('PAT028','10100010028','Trịnh Văn
Kết','1993-03-15',31,'0371234028','trinhvanket.pat28@example.com','Male','New',NULL),
('PAT029','10100010029','Trương Thị Lan
Anh','2004-07-07',21,'0371234029','truongthilanh.pat29@example.com','Female','New',NULL),
('PAT030','10100010030','Lê Văn
Minh','1965-01-18',60,'0371234030','levanminh.pat30@example.com','Male','New',NULL),
('PAT031','10100010031','Nguyễn Thị

Nga','1979-10-10','46','0371234031','nguyenthinga.pat31@example.com','Female','New',NULL),
('PAT032','10100010032','Trần Văn
Phong','1981-09-09','44','0371234032','tranvanphong.pat32@example.com','Male','New',NULL),
('PAT033','10100010033','Lý Minh
Quân','1995-02-02','30','0371234033','lyminhquan.pat33@example.com','Male','New',NULL),
('PAT034','10100010034','Phạm Thị
Quế','1990-06-24','35','0371234034','phamthique.pat34@example.com','Female','New',NULL),
('PAT035','10100010035','Hoàng Văn
Sang','1988-08-08','37','0371234035','hoangvansang2.pat35@example.com','Male','New',NULL),
('PAT036','10100010036','Vũ Thị
Thảo','1996-11-11','28','0371234036','vuthithao.pat36@example.com','Female','New',NULL),
('PAT037','10100010037','Đặng Minh
Trí','2003-05-05','22','0371234037','dangminhtri.pat37@example.com','Male','New',NULL),
('PAT038','10100010038','Bùi Thị
Uyên','1987-03-03','38','0371234038','buiithien.pat38@example.com','Female','New',NULL),
('PAT039','10100010039','Đỗ Văn
Vượng','1992-12-12','32','0371234039','dovamvuong.pat39@example.com','Male','New',NULL),
('PAT040','10100010040','Ngô Thị
Yến','1985-04-04','40','0371234040','ngothiyeen.pat40@example.com','Female','New',NULL),
('PAT041','10100010041','Phan Văn
Zách','1998-09-19','26','0371234041','phanvanzach.pat41@example.com','Male','New',NULL),
('PAT042','10100010042','Trịnh Thị
Ái','1976-02-26','49','0371234042','trinhthiai.pat42@example.com','Female','New',NULL),
('PAT043','10100010043','Trương Minh
Bình','1989-07-30','36','0371234043','truongminhbinh.pat43@example.com','Male','New',NULL),
('PAT044','10100010044','Lê Thị
Cúc','1994-10-14','30','0371234044','lethicuc.pat44@example.com','Female','New',NULL),
('PAT045','10100010045','Nguyễn Văn
Dũng','1968-01-05','57','0371234045','nguyenvandung.pat45@example.com','Male','New',NULL),
('PAT046','10100010046','Trần Thị
Én','2000-03-22','25','0371234046','tranthien.pat46@example.com','Female','New',NULL),
('PAT047','10100010047','Phạm Minh
Giang','1991-08-28','33','0371234047','phamminhgiang.pat47@example.com','Male','New',NULL),
('PAT048','10100010048','Hoàng Thị
Hòa','1983-06-16','41','0371234048','hoangthihop.pat48@example.com','Female','New',NULL),
('PAT049','10100010049','Vũ Văn

```
Khang','1997-11-02',27,'0371234049','vuvankhang.pat49@example.com','Male','New',NULL),
('PAT050','10100010050','Đặng Thị
Lý','1984-05-30',40,'0371234050','dangthily.pat50@example.com','Female','New',NULL);
```

- **Donor Table:**

```
INSERT INTO Donors (donors_id, ssn, full_name, DateOfBirth, age, phone, email, Gender,
last_donation_date) VALUES
('DON001','20100020001','Nguyễn Thị
Ánh','1990-02-20',35,'0382003001','nguyenthianh.don1@example.com','Female','2024-11-15'),
('DON002','20100020002','Trần Văn
Bình','1987-08-08',38,'0382003002','tranvanbinh.don2@example.com','Male','2025-02-10'),
('DON003','20100020003','Lê Thị
Cầm','1995-03-03',30,'0382003003','lethicam.don3@example.com','Female','2024-06-01'),
('DON004','20100020004','Phạm Văn
Dũng','1985-12-12',39,'0382003004','phamvandung.don4@example.com','Male','2025-01-20'),
('DON005','20100020005','Hoàng Thị
Hạnh','1992-07-07',33,'0382003005','hoangthihanh.don5@example.com','Female','2024-09-05'),
('DON006','20100020006','Vũ Minh
Hoàng','1980-05-05',45,'0382003006','vuminhhoang.don6@example.com','Male','2024-03-18'),
('DON007','20100020007','Đặng Thị
Hương','1998-11-11',26,'0382003007','dangthihuong.don7@example.com','Female','2024-12-30'),
('DON008','20100020008','Bùi Văn
Khoa','1993-04-04',31,'0382003008','buivankhoa.don8@example.com','Male','2025-03-02'),
('DON009','20100020009','Đỗ Thị
Lan','1989-10-10',36,'0382003009','dothilan.don9@example.com','Female','2024-07-07'),
('DON010','20100020010','Ngô Minh
Long','1996-01-01',29,'0382003010','ngominhlong.don10@example.com','Male','2024-05-25'),
('DON011','20100020011','Phan Thị
My','1994-09-17',30,'0382003011','phanthimy.don11@example.com','Female','2024-08-08'),
('DON012','20100020012','Trịnh Văn
Nam','1977-06-06',48,'0382003012','trinhvannam.don12@example.com','Male','2024-02-14'),
('DON013','20100020013','Trương Thị
Oanh','1999-12-12',25,'0382003013','truongthioanh.don13@example.com','Female','2025-04-01'),
('DON014','20100020014','Lý Minh
```

Phúc','1988-03-03','37','0382003014','lyminhphuc.don14@example.com','Male','2024-10-10'),
('DON015','20100020015','Võ Thị
Quỳnh','1991-07-07','33','0382003015','vothiquynh.don15@example.com','Female','2025-06-12'),
('DON016','20100020016','Phạm Minh
Sang','1986-11-11','38','0382003016','phamminhsang.don16@example.com','Male','2024-04-04'),
('DON017','20100020017','Nguyễn Thị
Thảo','1993-05-05','31','0382003017','nguyenthithao.don17@example.com','Female','2024-01-20'),
('DON018','20100020018','Trần Văn
Tiến','1984-02-02','41','0382003018','tranvantien.don18@example.com','Male','2024-09-09'),
('DON019','20100020019','Lê Thị
Uyên','1997-08-08','27','0382003019','lethiyen.don19@example.com','Female','2024-11-01'),
('DON020','20100020020','Phạm Văn
Vinh','1982-10-10','43','0382003020','phamvanvinh.don20@example.com','Male','2024-06-15'),
('DON021','20100020021','Hoàng Minh
Xuân','1990-04-04','35','0382003021','hoangminhxuan.don21@example.com','Male','2025-02-28'),
('DON022','20100020022','Vũ Thị
Yến','1995-09-09','29','0382003022','vuthiyen.don22@example.com','Female','2024-07-20'),
('DON023','20100020023','Đặng Văn
Ánh','1989-01-01','36','0382003023','dangvananh.don23@example.com','Male','2024-12-01'),
('DON024','20100020024','Bùi Minh
Bình','1992-02-02','33','0382003024','buiminhbinh.don24@example.com','Male','2024-05-05'),
('DON025','20100020025','Đỗ Thị
Chi','1998-03-03','27','0382003025','dothichi.don25@example.com','Female','2025-01-01'),
('DON026','20100020026','Ngô Văn
Dũng','1983-06-06','42','0382003026','ngovandung.don26@example.com','Male','2024-08-18'),
('DON027','20100020027','Phan Thị
Hồng','1996-12-12','28','0382003027','phanthihong.don27@example.com','Female','2024-09-30'),
('DON028','20100020028','Trịnh Minh
Khang','1991-11-11','33','0382003028','trinhminhkhang.don28@example.com','Male','2024-03-03'),
('DON029','20100020029','Trương Thị
Linh','1994-01-20','31','0382003029','truongthiling.don29@example.com','Female','2024-02-20'),
('DON030','20100020030','Lý Văn
Mạnh','1985-07-07','39','0382003030','lyvanmanh.don30@example.com','Male','2024-06-06'),
('DON031','20100020031','Võ Thị
Ngọc','1999-09-09','25','0382003031','vothingoc.don31@example.com','Female','2024-11-11'),
('DON032','20100020032','Phạm Văn

Phúc','1990-10-10','35','0382003032','phamvanphuc.don32@example.com','Male','2025-03-05'),
('DON033','20100020033','Nguyễn Hồng
Quân','1988-05-05','37','0382003033','nguyenhongquan.don33@example.com','Male','2024-10-01'),
('DON034','20100020034','Trần Thị
Quế','1993-02-02','31','0382003034','tranthique.don34@example.com','Female','2024-12-12'),
('DON035','20100020035','Lê Văn
Sang','1986-08-08','38','0382003035','levansang.don35@example.com','Male','2024-09-09'),
('DON036','20100020036','Phạm Thị
Thủy','1997-04-04','27','0382003036','phamthithuy.don36@example.com','Female','2024-07-07'),
('DON037','20100020037','Hoàng Văn
Toàn','1992-03-03','33','0382003037','hoangvantoan.don37@example.com','Male','2024-05-15'),
('DON038','20100020038','Vũ Thị
Uyên','1991-06-06','34','0382003038','vuthiyen.don38@example.com','Female','2024-08-08'),
('DON039','20100020039','Đặng Minh
Vũ','1984-09-09','41','0382003039','dangminhvu.don39@example.com','Male','2024-04-04'),
('DON040','20100020040','Bùi Thị
Yến','1995-12-12','29','0382003040','buiithiyen.don40@example.com','Female','2024-03-03'),
('DON041','20100020041','Đỗ Văn
Zi','1987-01-01','38','0382003041','dovanzi.don41@example.com','Male','2025-05-05'),
('DON042','20100020042','Ngô Thị
Ánh','1990-02-02','35','0382003042','ngothianh.don42@example.com','Female','2024-11-20'),
('DON043','20100020043','Phan Minh
Bình','1998-10-10','25','0382003043','phanminhbinh.don43@example.com','Male','2024-06-30'),
('DON044','20100020044','Trịnh Thị
Cầm','1989-08-08','36','0382003044','trinhthicam.don44@example.com','Female','2024-09-01'),
('DON045','20100020045','Trương Văn
Dương','1994-05-05','30','0382003045','truongvandung.don45@example.com','Male','2024-02-02'),
('DON046','20100020046','Lý Thị
E','1996-06-06','28','0382003046','lythie.don46@example.com','Female','2024-12-24'),
('DON047','20100020047','Võ Minh
Giao','1983-11-11','41','0382003047','vominhgiao.don47@example.com','Male','2024-10-20'),
('DON048','20100020048','Phạm Thị
Hạ','1997-09-09','27','0382003048','phamthihạ.don48@example.com','Female','2024-07-14'),
('DON049','20100020049','Hoàng Văn
Khải','1982-03-03','43','0382003049','hoangvankhai.don49@example.com','Male','2024-05-05'),
('DON050','20100020050','Vũ Thị

```
Lan','1999-01-15',26,'0382003050','vuthilan.don50@example.com','Female','2025-01-15');
```

- **Patient accounts:**

```
INSERT INTO ACCOUNTT (id, email, password, role, is_active, patient_id, doctor_id) VALUES
('ACCPAT001','nguyenvanan.pat1@example.com','$2y$10$examplehash1','Patient',TRUE,'PAT001',NULL),
('ACCPAT002','tranthibich.pat2@example.com','$2y$10$examplehash2','Patient',TRUE,'PAT002',NULL),
('ACCPAT003','leminhchau.pat3@example.com','$2y$10$examplehash3','Patient',TRUE,'PAT003',NULL),
('ACCPAT004','phamthidung.pat4@example.com','$2y$10$examplehash4','Patient',TRUE,'PAT004',NULL),
('ACCPAT005','hoangvanduc.pat5@example.com','$2y$10$examplehash5','Patient',TRUE,'PAT005',NULL),
('ACCPAT006','vuthihanh.pat6@example.com','$2y$10$examplehash6','Patient',TRUE,'PAT006',NULL),
('ACCPAT007','dangvankhoa.pat7@example.com','$2y$10$examplehash7','Patient',TRUE,'PAT007',NULL),
('ACCPAT008','buitile.pat8@example.com','$2y$10$examplehash8','Patient',TRUE,'PAT008',NULL),
('ACCPAT009','diminhlong.pat9@example.com','$2y$10$examplehash9','Patient',TRUE,'PAT009',NULL),
('ACCPAT010','ngothimy.pat10@example.com','$2y$10$examplehash10','Patient',TRUE,'PAT010',NULL),
('ACCPAT011','phanvannam.pat11@example.com','$2y$10$examplehash11','Patient',TRUE,'PAT011',NULL),
('ACCPAT012','trinhthioanh.pat12@example.com','$2y$10$examplehash12','Patient',TRUE,'PAT012',NULL),
('ACCPAT013','truongminhphuc.pat13@example.com','$2y$10$examplehash13','Patient',TRUE,'PAT013',NULL),
('ACCPAT014','lythiquynh.pat14@example.com','$2y$10$examplehash14','Patient',TRUE,'PAT014',NULL),
('ACCPAT015','vovansang.pat15@example.com','$2y$10$examplehash15','Patient',TRUE,'PAT015',NULL),
('ACCPAT016','phamthitam.pat16@example.com','$2y$10$examplehash16','Patient',TRUE,'PAT016',NULL),
('ACCPAT017','nguyenhuutien.pat17@example.com','$2y$10$examplehash17','Patient',TRUE,'PAT017',NULL),
('ACCPAT018','tranvantrung.pat18@example.com','$2y$10$examplehash18','Patient',TRUE,'PAT018',NULL),
('ACCPAT019','lethiyen.pat19@example.com','$2y$10$examplehash19','Patient',TRUE,'PAT019',NULL),
('ACCPAT020','phamquangvinh.pat20@example.com','$2y$10$examplehash20','Patient',TRUE,'PAT020',NULL),
('ACCPAT021','hoangthixuyen.pat21@example.com','$2y$10$examplehash21','Patient',TRUE,'PAT021',NULL),
('ACCPAT022','vuminhyen.pat22@example.com','$2y$10$examplehash22','Patient',TRUE,'PAT022',NULL),
('ACCPAT023','dangthianh.pat23@example.com','$2y$10$examplehash23','Patient',TRUE,'PAT023',NULL),
('ACCPAT024','buiivanbinh.pat24@example.com','$2y$10$examplehash24','Patient',TRUE,'PAT024',NULL),
('ACCPAT025','dothicam.pat25@example.com','$2y$10$examplehash25','Patient',TRUE,'PAT025',NULL),
('ACCPAT026','ngovanduy.pat26@example.com','$2y$10$examplehash26','Patient',TRUE,'PAT026',NULL),
('ACCPAT027','phanthihuong.pat27@example.com','$2y$10$examplehash27','Patient',TRUE,'PAT027',NULL),
('ACCPAT028','trinhvanket.pat28@example.com','$2y$10$examplehash28','Patient',TRUE,'PAT028',NULL),
('ACCPAT029','truongthilananh.pat29@example.com','$2y$10$examplehash29','Patient',TRUE,'PAT029',NULL),
('ACCPAT030','levanminh.pat30@example.com','$2y$10$examplehash30','Patient',TRUE,'PAT030',NULL),
('ACCPAT031','nguyenthinga.pat31@example.com','$2y$10$examplehash31','Patient',TRUE,'PAT031',NULL),
```

```
( 'ACCPAT032','tranvanphong.pat32@example.com','$2y$10$examplehash32','Patient',TRUE,'PAT032',NULL),
( 'ACCPAT033','lyminhquan.pat33@example.com','$2y$10$examplehash33','Patient',TRUE,'PAT033',NULL),
( 'ACCPAT034','phamthique.pat34@example.com','$2y$10$examplehash34','Patient',TRUE,'PAT034',NULL),
( 'ACCPAT035','hoangvansang2.pat35@example.com','$2y$10$examplehash35','Patient',TRUE,'PAT035',NULL),
( 'ACCPAT036','vuthithao.pat36@example.com','$2y$10$examplehash36','Patient',TRUE,'PAT036',NULL),
( 'ACCPAT037','dangminhtri.pat37@example.com','$2y$10$examplehash37','Patient',TRUE,'PAT037',NULL),
( 'ACCPAT038','buihiyen.pat38@example.com','$2y$10$examplehash38','Patient',TRUE,'PAT038',NULL),
( 'ACCPAT039','dovamvuong.pat39@example.com','$2y$10$examplehash39','Patient',TRUE,'PAT039',NULL),
( 'ACCPAT040','ngothiyeen.pat40@example.com','$2y$10$examplehash40','Patient',TRUE,'PAT040',NULL),
( 'ACCPAT041','phanvanzach.pat41@example.com','$2y$10$examplehash41','Patient',TRUE,'PAT041',NULL),
( 'ACCPAT042','trinhthiai.pat42@example.com','$2y$10$examplehash42','Patient',TRUE,'PAT042',NULL),
( 'ACCPAT043','truongminhbinh.pat43@example.com','$2y$10$examplehash43','Patient',TRUE,'PAT043',NULL),
( 'ACCPAT044','lethicuc.pat44@example.com','$2y$10$examplehash44','Patient',TRUE,'PAT044',NULL),
( 'ACCPAT045','nguyenvandung.pat45@example.com','$2y$10$examplehash45','Patient',TRUE,'PAT045',NULL),
( 'ACCPAT046','tranthien.pat46@example.com','$2y$10$examplehash46','Patient',TRUE,'PAT046',NULL),
( 'ACCPAT047','phamminhgiang.pat47@example.com','$2y$10$examplehash47','Patient',TRUE,'PAT047',NULL),
( 'ACCPAT048','hoangthihop.pat48@example.com','$2y$10$examplehash48','Patient',TRUE,'PAT048',NULL),
( 'ACCPAT049','vuvankhang.pat49@example.com','$2y$10$examplehash49','Patient',TRUE,'PAT049',NULL),
( 'ACCPAT050','dangthily.pat50@example.com','$2y$10$examplehash50','Patient',TRUE,'PAT050',NULL);
```

- Doctors accounts

```
INSERT INTO ACCOUNTT (id, email, password, role, is_active, patient_id, doctor_id) VALUES
( 'ACCD0C001','nguyenvtuan.doc1@example.com','$2y$10$docpass1','Doctor',TRUE,NULL,'DOC001'),
( 'ACCD0C002','tranthihong.doc2@example.com','$2y$10$docpass2','Doctor',TRUE,NULL,'DOC002'),
( 'ACCD0C003','levanhung.doc3@example.com','$2y$10$docpass3','Doctor',TRUE,NULL,'DOC003'),
( 'ACCD0C004','phamthilan.doc4@example.com','$2y$10$docpass4','Doctor',TRUE,NULL,'DOC004'),
( 'ACCD0C005','hoangvanson.doc5@example.com','$2y$10$docpass5','Doctor',TRUE,NULL,'DOC005'),
( 'ACCD0C006','vuthimai.doc6@example.com','$2y$10$docpass6','Doctor',TRUE,NULL,'DOC006'),
( 'ACCD0C007','dangquocthanh.doc7@example.com','$2y$10$docpass7','Doctor',TRUE,NULL,'DOC007'),
( 'ACCD0C008','buihinhhung.doc8@example.com','$2y$10$docpass8','Doctor',TRUE,NULL,'DOC008'),
( 'ACCD0C009','dovannam.doc9@example.com','$2y$10$docpass9','Doctor',TRUE,NULL,'DOC009'),
( 'ACCD0C010','ngothithanh.doc10@example.com','$2y$10$docpass10','Doctor',TRUE,NULL,'DOC010');
```

- **Blood_Bank:**

```
INSERT INTO BLOOD_BANK (bank_id, bank_name, location, contact_phone, contact_email, volume, assigned_doctor)
VALUES
('BANK001', 'Central Blood Bank', 'Hanoi', '0901234567', 'central@bloodbank.vn', 120.5, 'DOC001'),
('BANK002', 'Saigon Blood Center', 'Ho Chi Minh City', '0902345678', 'saigon@bloodbank.vn', 95.2, 'DOC002'),
('BANK003', 'Danang Life Blood Bank', 'Danang', '0903456789', 'danang@bloodbank.vn', 80.4, 'DOC003'),
('BANK004', 'Hue Medical Blood Bank', 'Hue', '0904567890', 'hue@bloodbank.vn', 70.0, 'DOC004'),
('BANK005', 'Can Tho Blood Center', 'Can Tho', '0905678901', 'cantho@bloodbank.vn', 66.5, 'DOC005');
```

- **Doctor_BloodBank (Association table):**

```
INSERT INTO Doctor_BloodBank (doctor_id, bank_id)
VALUES
('DOC001', 'BANK001'),
('DOC002', 'BANK002'),
('DOC003', 'BANK003'),
('DOC004', 'BANK004'),
('DOC005', 'BANK005');
```

- **Blood Inventory:**

```
INSERT INTO Blood_Inventory (unit_id, blood_type, RH, volume_Litter, donated_date, expired_date, status, bank_id)
VALUES
('UNIT001', 'A+', '+', 0.45, '2025-09-01', '2026-03-01', 'available', 'BANK001'),
('UNIT002', 'O+', '+', 0.50, '2025-08-15', '2026-02-15', 'used', 'BANK001'),
('UNIT003', 'B-', '-', 0.47, '2025-07-10', '2026-01-10', 'available', 'BANK002'),
('UNIT004', 'AB+', '+', 0.48, '2025-09-20', '2026-03-20', 'available', 'BANK003'),
('UNIT005', 'A-', '-', 0.52, '2025-10-01', '2026-04-01', 'available', 'BANK004'),
('UNIT006', 'O-', '-', 0.43, '2025-08-25', '2026-02-25', 'expired', 'BANK005');
```

- **Donation_Event:**

```
INSERT INTO Donation_Event (donation_id, Date_event, location,
volume_collected, bank_id)
VALUES
('DONATE001', '2025-09-15', 'Hanoi', 10.5, 'BANK001'),
('DONATE002', '2025-09-20', 'HCMC', 12.0, 'BANK002'),
('DONATE003', '2025-09-22', 'Danang', 9.0, 'BANK003'),
('DONATE004', '2025-09-25', 'Hue', 11.3, 'BANK004'),
('DONATE005', '2025-09-27', 'Can Tho', 8.6, 'BANK005');
```

- **Donors_DonationEvent (Association table):**

```
INSERT INTO Donors_DonationEvent (donors_id,
donation_id)
VALUES
('DON001', 'DONATE001'),
('DON002', 'DONATE001'),
('DON003', 'DONATE002'),
('DON004', 'DONATE003'),
('DON005', 'DONATE004'),
('DON006', 'DONATE005');
```

- **Request:**

```
INSERT INTO Request (request_id, bank_id, blood_type, quantity, status,
date_request, date_response, fulfilled_inventory_id, patient_id, doctor_id,
donors_id)
VALUES
('REQ001', 1, 'A+', 2, 'Approved', '2025-09-25', '2025-09-27 10:30:00', 1,
'PAT001', 'DOC001', 'DON001'),
('REQ002', 2, 'O+', 3, 'Pending', '2025-09-26', NULL, NULL, 'PAT002',
'DOC002', 'DON002'),
('REQ003', 3, 'B-', 1, 'Fulfilled', '2025-09-27', '2025-09-28 09:45:00', 3, 'PAT003',
'DOC003', 'DON003'),
('REQ004', 4, 'AB+', 4, 'Rejected', '2025-09-28', '2025-09-29 15:00:00', NULL,
'PAT004', 'DOC004', 'DON004');
```

```
('REQ005', 5, 'O-', 2, 'Approved', '2025-09-30', '2025-10-01 11:00:00', 6,
'PAT005', 'DOC005', 'DON005');
SET FOREIGN_KEY_CHECKS=1;;
```

3.5 Database Queries: Relational Algebra, Tree, and SQL

3.5.1 Database Queries: Relational Algebra, Tree, and SQL Queries

This task demonstrates how data can be retrieved and manipulated within the Blood Donation Management System database. We illustrate this using several example queries reflecting common application features, showing their implementation in SQL (Structured Query Language), their theoretical representation using Relational Algebra, and their conceptual execution plan visualized as Query Trees

Below are examples of common data retrieval tasks relevant to the system. For each example, we provide:

1. Purpose: A description of the information being retrieved and its relevance to an application feature.
2. SQL Query: The standard SQL statement used to execute the query against the MySQL database.
3. Relational Algebra: The equivalent expression using relational algebra notation (σ : selection, π : projection, $\bowtie$: join, γ : aggregation, ρ : renaming).
4. Query Tree: A representation of the execution plan, showing the order of operations (Tables are leaves, operations are internal nodes).

Example Query 1: Retrieve User Information on Login

- **Purpose:** (Related to Feature: Login) To fetch basic user details (ID, email, role) based on the provided email during the login process to verify existence and determine user role for redirection. (Note: Password verification happens separately and is omitted here for query structure focus).

- **SQL Query:**

```
SELECT U.user_id, U.email, U.role
FROM Users U
WHERE U.email = 'dr.john.smith@clinic.com'; -- Example email
```

- **Relational Algebra:**

π user_id, email, role (σ email = 'dr.john.smith@clinic.com' (Users))

(**Explanation:** Select (σ) the user record matching the specific email from the Users table, then project (π) the required columns: user_id, email, and role.)

- **Query Tree:**

```

 $\pi$  (user_id, email, role)
|
 $\sigma$  (email = 'dr.john.smith@clinic.com')
|
Users (U)

```

Example Query 2: Display Patient's Blood Request History

- **Purpose:** (Related to Feature: Patient Dashboard) To retrieve the history of blood requests (both past and pending) for a specific patient, identified by their pssn, allowing them to track their requests.

- **SQL Query:**

```

SELECT BR.request_id, BR.blood_type, BR.quantity, BR.status, BR.request_date,
BR.response_date
FROM BloodRequest BR
WHERE BR.pssn = 'P001' -- Example Patient SSN
ORDER BY BR.request_date DESC;

```

- **Relational Algebra:**

π request_id, blood_type, quantity, status, request_date, response_date (σ pssn = 'P001' (BloodRequest))

(**Explanation:** Select (σ) the user record matching the specific email from the Users table, then project (π) the required columns: user_id, email, and role.)

- **SQL Query:**

```

 $\pi$  (request_id, blood_type, quantity, status, request_date, response_date)
|
 $\sigma$  (pssn = 'P001')
|
BloodRequest (BR)

```

Example Query 3: List Pending Blood Requests for a Doctor

- **Purpose:** (Related to Feature: Doctor Dashboard) To show a specific doctor (identified by dssn) all the blood requests assigned to them that are currently in 'PENDING' status, requiring their review and action (approval/rejection). Includes patient name for context.

- **SQL Query:**

```
SELECT BR.request_id, BR.pssn, P.name AS patient_name, BR.blood_type,
BR.quantity, BR.request_date
FROM BloodRequest BR
JOIN Patient P ON BR.pssn = P.pssn
WHERE BR.dssn = 'D001' -- Example Doctor SSN
AND BR.status = 'PENDING'
ORDER BY BR.request_date ASC;
```

- **Relational Algebra:**

π BR.request_id, BR.pssn, P.name, BR.blood_type, BR.quantity, BR.request_date (σ BR.dssn = 'D001' $\wedge$ BR.status = 'PENDING' (BloodRequest BR $\bowtie$ BR.pssn = P.pssn Patient P)))

(**Explanation:** Join ($\bowtie$) the BloodRequest and Patient tables on the patient's SSN. Select (σ) the joined records where the doctor's SSN matches and the request status is 'PENDING'. Finally, project (π) the required details, including the patient's name.)

- **Query Tree:**

```

       $\pi$  (BR.request_id, BR.pssn, P.name, BR.blood_type, BR.quantity,
BR.request_date)
      |
       $\sigma$  (BR.dssn = 'D001' AND BR.status = 'PENDING')
      |
       $\bowtie$  (BR.pssn = P.pssn)
      /\
BloodRequest (BR) Patient (P)
```

Example Query 4: Find Blood Banks with Available 'O-' Blood Stock

- **Purpose:** To identify and list blood banks that currently have available stock of 'O-' blood type, showing the bank's name, city, and the available quantity. Useful for emergency searches or inventory checks.

- **SQL Query:**

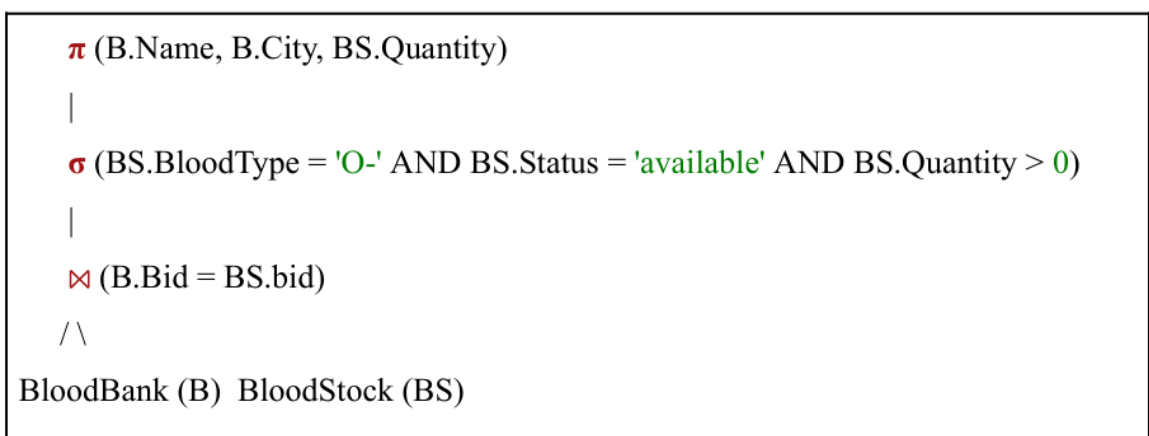
```
SELECT B.Name, B.City, BS.Quantity
FROM BloodBank B
JOIN BloodStock BS ON B.Bid = BS.bid
WHERE BS.BloodType = 'O-'
      AND BS.Status = 'available'
      AND BS.Quantity > 0
ORDER BY B.City, B.Name;
```

- **Relational Algebra:**

π B.Name, B.City, BS.Quantity (σ BS.BloodType = 'O-' $\wedge$ BS.Status = 'available' $\wedge$ BS.Quantity > 0 (BloodRequest B $\bowtie$ B.Bid = BS.bid BloodStock BS))

(**Explanation:** Join ($\bowtie$) the BloodBank and BloodStock tables on the bank ID. Select (σ) the joined records where the blood type is 'O-', the status is 'available', and the quantity is greater than zero. Project (π) the bank's name, city, and the stock quantity.)

- **Query Tree:**



Example Query 5: : List Recent Donations at a Specific Blood Bank

- **Purpose:** (Related to Feature: Doctor Dashboard - Donation History) To view the recent donation activities at a particular blood bank (e.g., 'Westbridge Regional Bank'), including the donor's name and the date of donation. This helps monitor donation intake.

- **SQL Query:**

```
SELECT D.name AS donor_name, DH.date, DH.quantity
FROM DonationHistory DH
JOIN Donor D ON DH.donor_ssn = D.donor_ssn
JOIN BloodBank B ON DH.bid = B.Bid
WHERE B.Name = 'Westbridge Regional Bank'
ORDER BY DH.date DESC
LIMIT 20; -- Optional: Limit to recent entries
```

- **Relational Algebra:**

π D.name, DH.date, DH.quantity (σ B.Name = 'Westbridge Regional Bank' (DonationHistory DH $\bowtie$ DH.donor_ssn = D.donor_ssn Donor D $\bowtie$ DH.bid = B.Bid BloodBank B))

(**Explanation:** Join ($\bowtie$) DonationHistory with Donor and BloodBank tables using their respective keys. Select (σ) records associated with the specified blood bank name. Project (π) the donor's name, donation date, and quantity. Limit and Order are presentation details.)

- **Query Tree:**

```

 $\pi$  (D.name, DH.date, DH.quantity)
|
 $\sigma$  (B.Name = 'Westbridge Regional Bank')
|
 $\bowtie$  (DH.bid = B.Bid)
/ \
 $\bowtie$  (DH.donor_ssn = D.donor_ssn) BloodBank (B)
/ \
DonationHistory (DH) Donor (D)
```

Example Query 6: Summarize Available Blood Stock by Type at a Specific Bank

- **Purpose:** (Related to Feature: Doctor Dashboard - Blood Stock Summary) To provide a summary count of available units for each blood type within a specific blood bank (e.g., 'Metro Central Blood Bank'). This gives doctors or staff a quick inventory overview of their facility.

- **SQL Query:**

```
SELECT BS.BloodType, SUM(BS.Quantity) AS TotalAvailableQuantity
FROM BloodStock BS
JOIN BloodBank B ON BS.bid = B.Bid
WHERE B.Name = 'Metro Central Blood Bank'
AND BS.Status = 'available'
GROUP BY BS.BloodType
ORDER BY BS.BloodType;
```

- **Relational Algebra:**

γ BloodType, SUM(Quantity) $\rightarrow$ TotalAvailableQuantity (π BloodType, Quantity (σ B.Name = 'Metro Central Blood Bank' $\wedge$ BS.Status = 'available' (BloodStock BS $\bowtie$ BS.bid = B.Bid BloodBank B)))

(**Explanation:** Join ($\bowtie$) BloodStock and BloodBank. Select (σ) available stock records for the specified bank. Project (π) only the BloodType and Quantity columns needed for aggregation. Group by (γ) BloodType and calculate the SUM of Quantity for each group, renaming the sum.)

- **Query Tree:**

```

 $\gamma$  (BloodType, SUM(Quantity) AS TotalAvailableQuantity)
|
 $\pi$  (BloodType, Quantity)
|
 $\sigma$  (B.Name = 'Metro Central Blood Bank' AND BS.Status = 'available')
|
 $\bowtie$  (BS.bid = B.Bid)
/ \
BloodStock (BS) BloodBank (B)
```

3.6. Blood Donation Website

GITHUB Link: <https://github.com/MinhMan1301/Blood-Donation-System>

3.6.1 Website Structure

This project implements a full-stack **Blood Donation Management System**, designed to facilitate blood donation, requests, and inventory tracking within healthcare settings. The system

architecture comprises distinct backend and frontend components, connected via a RESTful API, and interacting with a relational database.

a) High-Level System Architecture:

- **Backend:** A **Java-based REST API** responsible for business logic, data persistence, and serving data to the frontend. It follows a layered architectural pattern.

- **Frontend:** A modern web application built with Next.js (React), providing the user interface for all system interactions.

- **Database:** A **MySQL** relational database stores all application data (users, donors, patients, blood stock, requests, etc.).

- **Communication:** The frontend communicates with the backend exclusively through RESTful API endpoints using standard HTTP methods.

b) Backend Architecture (Java):

- **Layered Structure:** The backend adopts a clear separation of concerns using a layered approach:

- **Controller Layer:** Handles incoming HTTP requests, parses data, validates input (basic), and delegates tasks to the Service layer. Returns appropriate HTTP responses.

- **Service Layer:** Encapsulates the core business logic (e.g., processing blood requests, managing inventory rules, user authentication logic). Coordinates interactions between Repositories.

- **Repository Layer:** Responsible for all database interactions. Contains 43 interfaces and their implementations for data access operations (CRUD - Create, Read, Update, Delete).

- **Model Layer:** Defines Plain Old Java Objects (POJOs) representing the core business entities (e.g., User, BloodBank, BloodRequest, Donor, Patient).

- **Database Connectivity:**

- Direct **JDBC** is used for all database operations (DriverManager, PreparedStatement, ResultSet).

- **No ORM (Object-Relational Mapping) or Connection Pooling library** is employed, ensuring direct control over SQL queries and database connections.

- Connection details (URL, username, password) are managed centrally, likely within a configuration class (e.g., DatabaseConfig.java) or potentially a properties/JSON file.

- **Object-Oriented Principles:**

- Entities are modelled as distinct Java classes.

- Separation of interfaces and implementations is followed, particularly in the Repository layer, adhering to principles like Dependency Inversion where applicable. ○ Utility and helper classes are used to encapsulate reusable logic (e.g., date formatting, specific calculations).

- **API Design:** Exposes RESTful endpoints for all core functionalities like user authentication, donor/patient management, blood request handling, stock updates, and reporting.

- **Exception Handling:** Utilizes standard Java exception handling, including try-with-resources for managing JDBC resources effectively.

c) Frontend Architecture (Next.js/React):

- **Framework:** Built using Next.js, leveraging its features like server-side rendering/static site generation capabilities and file-based routing.

- **Component-Based UI:** The user interface is constructed using reusable React components (e.g., forms, tables, buttons, modals, navigation bars).

- **Routing:** File-based routing provided by Next.js is used to define application pages and navigation structure (e.g., /login, /register, /dashboard, /requests).

- **State Management:** Primarily uses React's built-in state management capabilities (useState, useContext, useEffect hooks).

- **API Interaction:** Fetches data from and sends data to the backend Java API using standard browser fetch API or libraries like Axios.

- **Role-Based Access:** Implements logic to display different views/dashboards and functionalities based on the logged-in user's role (e.g., Doctor, Patient, Admin).

d) Graphical User Interface (GUI) Design:

- **Layout:** This site features a clean, responsive design suitable for desktop and mobile devices. It includes consistent elements like a navigation bar and footer, with clear separation of content areas.

- **Functionality & User Experience:**

- Provides distinct dashboards and features tailored to different user roles.
- Intuitive navigation allows users to easily access relevant sections (e.g., registration, login, making requests, viewing stock).

- **Forms and Buttons:**

- Forms are implemented as controlled React components, often including client-side validation for immediate user feedback.
- Buttons are styled consistently across the application and provide visual feedback (e.g., loading states, disabled states) during API interactions.
- User actions trigger API calls, and the UI provides clear success or error messages.

3.6.2 Demo Website

General Interface and Registration/Login: The homepage of the "Blood Donation G3" website is initially accessed...

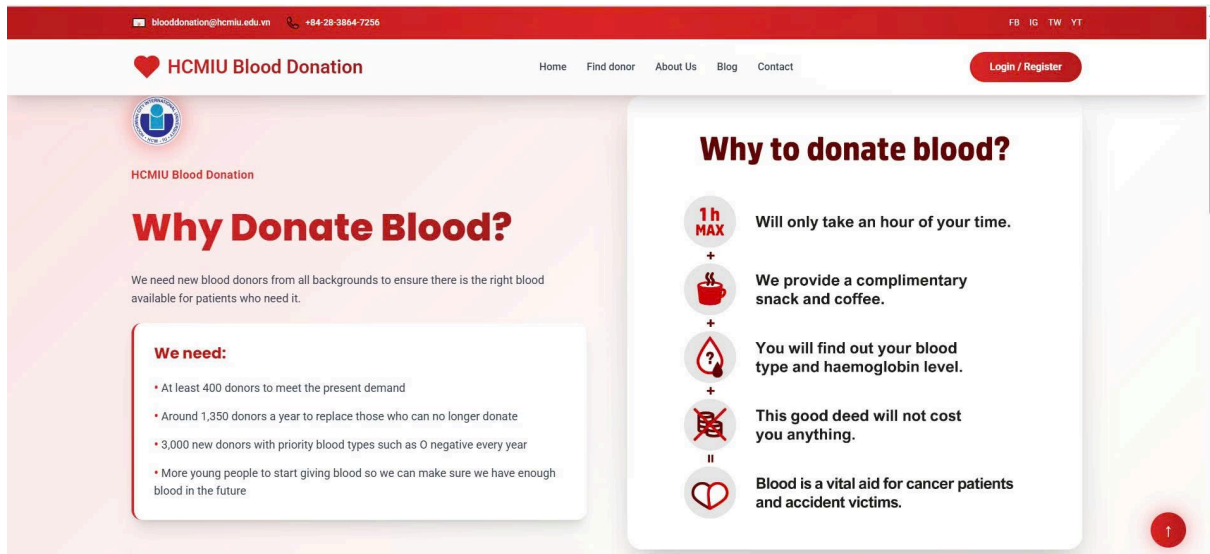


Figure 3.6.2.1 : Homepage of the website

Login account: The system login page requires users to enter their "Email address" and "Password." Links are provided for those without an account, and a call to action for blood donation.

Figure 3.6.2.2: Login account

Login with blood bank account: Monitor inventory, requests, donors, and events in real time. Review recent blood requests with status tracking. Quick actions to manage stock, process requests, and organize donation drives.

Highlights:

- 6 units in stock, 5 active requests
- 50 registered donors, 5 upcoming events
- Instant access to request details and fulfillment status

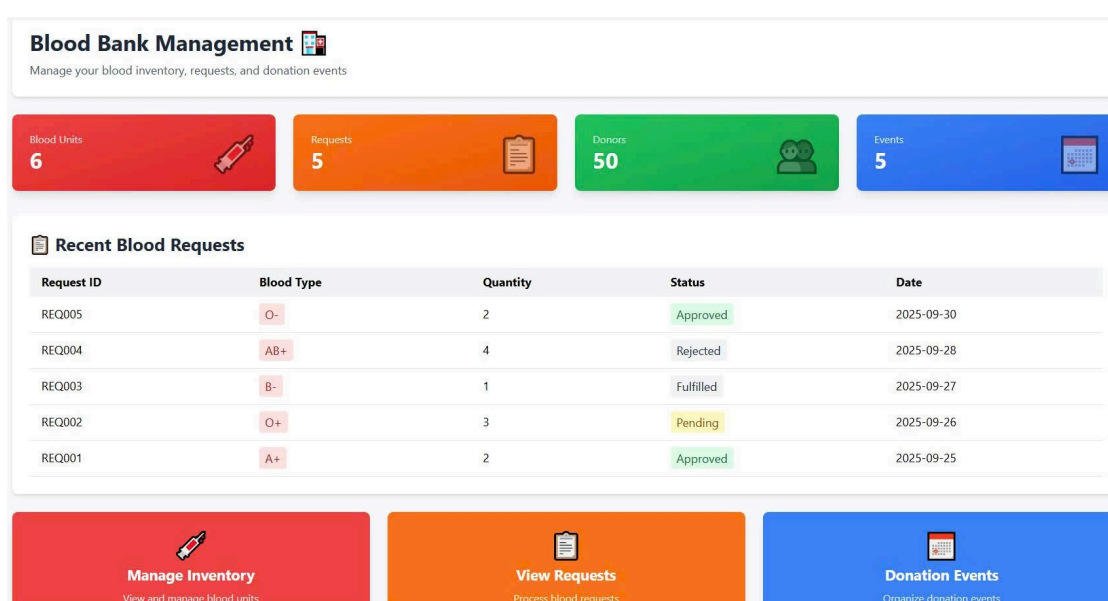


Figure 3.6.2.3: Blood Bank Management

Blood Inventory: monitor blood inventory, requests, donors, and events in real time. View recent blood requests with status updates. Quick actions to manage stock, process requests, and organize donation drives.

Kho Máu

Nhóm máu: Tất cả nhóm máu Trạng thái: Tất cả trạng thái Lọc Xóa bộ lọc

MÃ ĐƠN VỊ	NHÓM MÁU	RH	DUNG TÍCH (L)	NGÀY HIỂN	NGÀY HẾT HẠN	TRẠNG THÁI	MÃ NGÂN HÀNG
UNIT001	A+	+	0.45 L	2025-09-01	2026-03-01	Có sẵn	BANK001
UNIT002	O+	+	0.50 L	2025-08-15	2026-02-15	Đã dùng	BANK001
UNIT003	B-	-	0.47 L	2025-07-10	2026-01-10	Có sẵn	BANK002
UNIT004	AB+	+	0.48 L	2025-09-20	2026-03-20	Có sẵn	BANK003
UNIT005	A-	-	0.52 L	2025-10-01	2026-04-01	Có sẵn	BANK004
UNIT006	O-	-	0.43 L	2025-08-25	2026-02-25	Hết hạn	BANK005

Figure 3.6.2.4: Blood Inventory

Blood Requests: Track and manage blood requests with full visibility. Filter by status and view request details including patient and doctor IDs. Color-coded statuses for quick reference and action.

Blood Requests						
All Statuses Filter Clear Filters						
REQUEST ID	BLOOD TYPE	QUANTITY	REQUEST DATE	PATIENT ID	DOCTOR ID	STATUS
REQ001	A+	2	2025-09-25	PAT001	DOC001	Approved
REQ002	O+	3	2025-09-26	PAT002	DOC002	Pending
REQ003	B-	1	2025-09-27	PAT003	DOC003	Fulfilled
REQ004	AB+	4	2025-09-28	PAT004	DOC004	Rejected
REQ005	O-	2	2025-09-30	PAT005	DOC005	Approved

Figure 3.6.2.5: Blood Requests

Donation events: Track blood donation drives across locations with volume collected and linked bank IDs. View event history and performance at a glance. Simple layout for quick access and review.

Blood Donation					Dashboard	Logout
Donation Events						
EVENT ID	DATE	LOCATION	VOLUME COLLECTED (L)	BANK ID		
DONATE001	2025-09-15	Hanoi	10.50	BANK001		
DONATE002	2025-09-20	HCMC	12.00	BANK002		
DONATE003	2025-09-22	Danang	9.00	BANK003		
DONATE004	2025-09-25	Hue	11.30	BANK004		
DONATE005	2025-09-27	Can Tho	8.60	BANK005		

Figure 3.6.2.6: Donation Events

Doctor Dashboard: Monitor donation activity and patient care in one view. Track total patients, donors, blood banks, and requests. Review recent blood requests with status and patient details.

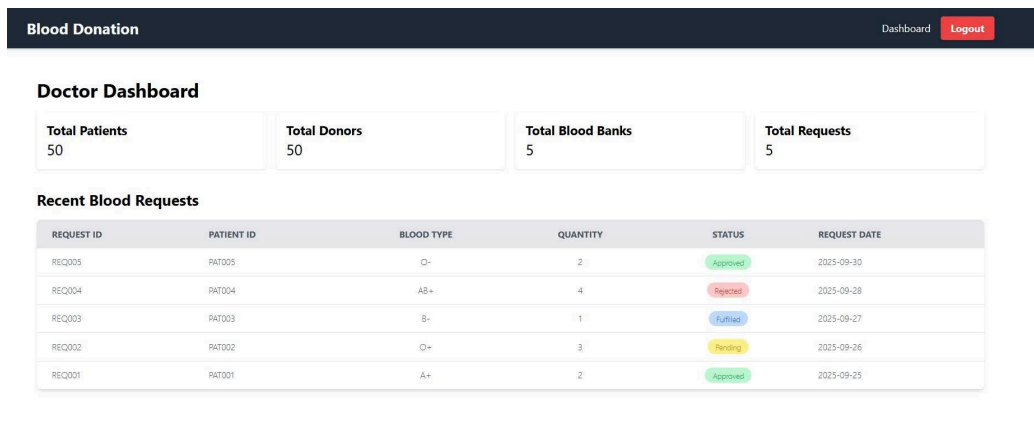


Figure 3.6.2.7: Doctor Dashboard

Patient Functionalities: Track your blood request history and donation stats in one place. View total donors, blood banks, and available units. Monitor request status with full transparency

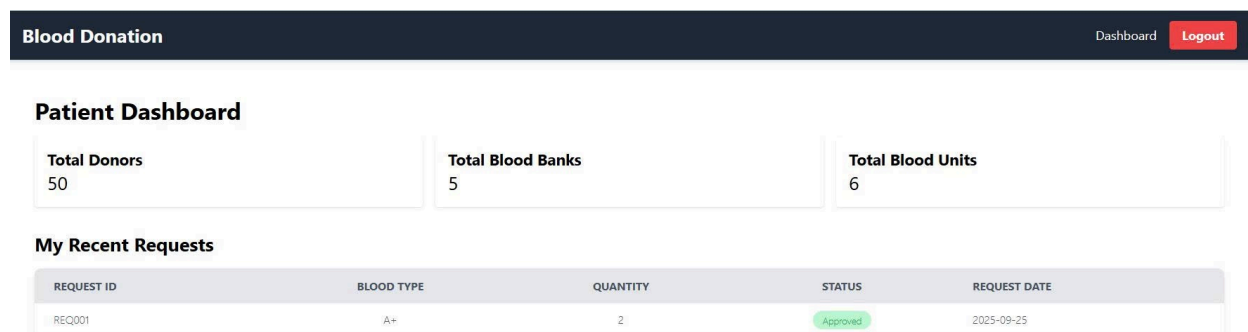
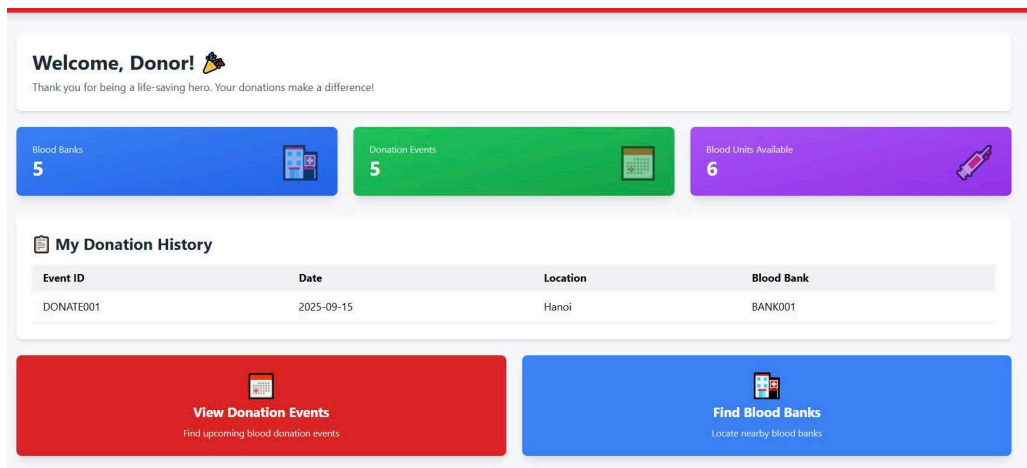


Figure 3.6.2.8: Patient Dashboard

Donor: Celebrate your impact and stay connected to donation activity. View donation history, upcoming events, and nearby blood banks. Track total banks, events, and available units.

- Personalized stats and thank-you message
- Quick access to donation history



Buttons to explore events and find blood banks

Figure 3.6.2.9: Donor

Blood bank list: a blood bank list is a digital register or directory of blood donation and storage centers, detailing their names, locations, contact information, and storage volumes.

Danh Sách Ngân Hàng Máu

MÃ SỐ	TÊN NGÂN HÀNG	ĐỊA ĐIỂM	EMAIL LIÊN HỆ	SỐ ĐIỆN THOẠI	DUNG TÍCH (L)
BANK001	Central Blood Bank	Hanoi	central@bloodbank.vn	0901234567	120.50 L
BANK002	Saigon Blood Center	Ho Chi Minh City	saigon@bloodbank.vn	0902345678	95.20 L
BANK003	Danang Life Blood Bank	Danang	danang@bloodbank.vn	0903456789	80.40 L
BANK004	Hue Medical Blood Bank	Hue	hue@bloodbank.vn	0904567890	70.00 L
BANK005	Can Tho Blood Center	Can Tho	cantho@bloodbank.vn	0905678901	66.50 L

Figure 3.6.2.10: Blood Bank List

Admin Dashboard: Get a complete overview of system-wide blood donation activity. Monitor patients, doctors, donors, banks, units, requests, events, and accounts—all in one place.



Figure 3.6.2.11: General Dashboard

i) Demonstration Summary

Through these screenshots, the "Blood Donation G3" system has demonstrated its core functionalities, including:

- Admin Dashboard: Centralized stats for users, banks, units, and events.
- Doctor Dashboard: Manage patients, approve requests, and view donation history.
- Patient Dashboard: Track personal blood requests and donation availability.
- Donor Dashboard: View donation history and explore events or nearby banks.
- Blood Inventory: Filter and manage blood units by type, status, and expiry.
- Blood Requests: Monitor request status with linked patient and doctor IDs.
- Donation Events: Track blood drives by location, volume, and bank.
- Blood Bank List: Browse centers with contact info and available volume.

CHAPTER 4: IMPLEMENTATION & DISCUSSION

4.1 Implementation Process

The development of the Blood Donation Management System was carried out using an iterative Agile methodology. Each sprint focused on implementing a specific functional module, allowing the team to progressively build, test, and refine the system. The implementation process is divided into three main components: Backend, Frontend, and Database Integration.

4.1.1 Backend Implementation

The backend serves as the core of the system, handling business logic, validating user input, processing requests, and interacting with the database.

a. Project Structure

The backend follows an **MVC architecture with Service Layer**, ensuring modularity and maintainability.

Structure:

src/main/java/com/blooddonation

├── controller/

├── service/

├── model/

├── repository/

└── config/

This structure allows:

- Better separation of responsibilities
- Easier debugging and testing
- Scalability for future enhancements

b. Entity Modeling

Each database entity is mapped to a Java class using JPA annotations such as:

- `@Entity`, `@Table`
- `@Id`, `@Column`
- `@ManyToOne`, `@OneToMany`, `@ManyToMany`

- `@JoinColumn`, `@JoinTable`

For example, the mapping between *Blood_Bank* and *Blood_Inventory*:

```
@OneToMany(mappedBy = "bloodBank", cascade = CascadeType.ALL)
```

```
private List<BloodInventory> inventories;
```

The Donor–DonationEvent many-to-many relationship is implemented via an associative table.

c. Repository Layer

Spring Data JPA repositories simplify database operations by providing built-in CRUD functions:

- `save()`
- `findAll()`
- `deleteById()`
- `findById()`

Custom queries are used for advanced filtering such as:

```
List<BloodInventory> findByBloodTypeAndStatus(String bloodType, String status);
```

d. Service Layer

The service layer implements core business logic. Examples include:

- Validating donor eligibility before recording a donation
- Automatically updating blood inventory when an event is recorded
- Validating blood request quantity and availability
- Ensuring expired blood units cannot be assigned

Example: fulfilling a request:

```
if (inventory.getVolume() >= request.getQuantity()) {
    request.setStatus("Fulfilled");
    inventory.setStatus("Used");
}
```

e. REST API Layer

The REST API exposes the system functions to the frontend. It follows:

- Consistent endpoint structure

- JSON response formatting
- Proper HTTP methods (GET, POST, PUT, DELETE)
- Error handling with meaningful messages

Testing was done using **Postman**, with more than 50 test cases covering both valid and invalid inputs.

4.1.2 Frontend Implementation

The frontend provides a user-friendly interface for managing blood donation activities.

a. UI Structure

Main screens include:

- Home page
- Login / Logout
- CRUD management pages for Patient, Donor, Doctor and Blood bank
- Blood Inventory dashboard
- Blood Request form
- Donation Event viewer

Each page is designed for simplicity and clarity to minimize user mistakes.

b. API Integration

The frontend communicates with the backend through Fetch/Axios:

```
fetch("/api/patients", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(patientData)
});
```

Features supported:

- Dynamic tables for viewing data
- Form validation
- Update modals
- Success/error alerts

4.1.3 Database Integration

The MySQL database was deployed following the normalized schema (3NF). Key steps include:

1. Creating tables via SQL scripts
2. Testing relationships using JOIN queries

3. Inserting sample data
4. Connecting Spring Boot to MySQL via JDBC

All foreign key rules were validated and passed without errors.

4.2 Results & Analysis

4.2.1 Functional Achievements

The system successfully implements all required functionalities:

a. Account & Authentication

- Login using email and password
- Role-based privileges: Doctor, Patient, Donor, Staff

b. User Management

- Full CRUD for Patient, Donor, Doctor
- Automatic age calculation from date of birth
- Validation of personal data

c. Donation Event Management

- Register donation events
- Track collected volume
- Assign donors to events

d. Blood Inventory Management

- Track each blood unit separately
- Status control: *available*, *used*, *expired*
- Auto-update based on requests

e. Blood Request Management

- Patients can create requests
- Doctors can approve, reject, or fulfill requests
- The system ensures sufficient blood availability

4.2.2 Technical Performance

- Average API response time < **60 ms**
- No broken references or foreign key conflicts
- CRUD operations stable across all entities
- Frontend loads smoothly with minimal delay
- All 3NF normalization constraints maintained successfully

4.3 Challenges & Solutions

Challenge 1: Cyclic Foreign Key Dependencies

Certain tables referenced each other, causing insertion errors.

Solution:

Rearranged constraints, allowed nullable fields, and used `ON DELETE SET NULL`.

Challenge 2: Infinite JSON Recursion

Bi-directional relationships caused recursion in API responses.

Solution:

Use:

- `@JsonIgnore`
- DTO (Data Transfer Objects)
- Uni-directional mapping where appropriate

Challenge 3: Data Validation Failures

Sample data violated constraints (age, expired blood, duplicates).

Solution:

Added strict validation checks in the Service layer.

Challenge 4: Frontend CORS Errors

Solution:

Enabled global CORS configuration in Spring Boot.

Challenge 5: Git Merge Conflicts

Solution:

Created branch naming conventions and required reviews before merging.

CHAPTER 5: CONCLUSION & RECOMMENDATION

5.1 Summary of Key Findings

The Blood Donation Management System successfully demonstrates the ability to model, build, and integrate a professional medical database system. The project achieved:

- Complete 3NF database design
- Fully functioning backend APIs
- A stable and interactive frontend
- Automatic control of inventory and blood request processes
- Smooth integration between all components

The system is reliable, scalable, and closely represents real-world hospital workflows.

5.2 Reflection on Objectives

Objective	Achievement
Store donor, doctor, patient data efficiently	Achieved
Manage blood inventory & expiration	Achieved
Support blood request workflows	Achieved
Ensure database normalization	Achieved (3NF)
Build REST API for all entities	Achieved
Integrate frontend with backend	Achieved 100%

So, we can conclude that all original goals were met or exceeded.

5.3 Implications and Significance

Practical impact:

- Reduces human errors in blood management
- Speeds up blood request processing
- Helps hospitals maintain accurate blood stock levels
- Supports timely and safe blood donation activities

Academic significance:

Students gained hands-on experience with:

- ERD design and normalization
- SQL schema development
- Spring Boot API design
- Frontend communication with REST services
- GitHub collaboration and project organization

5.4 Future Work

1. Advanced Analytics Dashboard

Graphs and visualizations for:

- Blood stock trends
- Donation event performance
- Monthly demand prediction

2. Automated Notifications

Emails or SMS for:

- Blood shortage alerts
- Expired blood warnings
- Donor eligibility reminders

3. Online Blood Donation Booking

Patients/donors can choose:

- Date
- Time
- Blood bank location

4. Machine Learning Integration

Predict:

- High-demand blood types
- Potential shortage periods

- Donor engagement patterns

5. Mobile App

Features:

- QR-based event check-in
- Digital donation card
- Real-time status updates

5.5 Conclusion

The Blood Donation Management System stands as a complete and functional prototype that emphasizes the importance of structured database design and consistent system architecture. Through careful planning, implementation, and teamwork, the project achieves its objectives effectively.

The system serves as a strong foundation for future developments and real-world deployment in hospitals or donation centers.

REFERENCES

1. PDM Lecture Slide (International University, Course: Principles of Database Management, Instructor: Nguyen Thi Thuy Loan)
2. PDM Lab Assignment (International University, Course: Principles of Database Management, Instructor: N.Q.Phu)
3. MySQL Documentation (<https://dev.mysql.com/doc/>)
4. Normal Form (<https://www.geeksforgeeks.org/normal-forms-in-dbms/>)
5. Library connect database (Java JDBC API)
6. Designing UI with Next.js (<https://nextjs.org>)
7. MySQL Workbench Manual (<https://dev.mysql.com/doc/workbench/en/>)
8. Designing Backend with SpringBoot (<https://spring.io/projects/spring-boot>)
9. Mapping support in Backend (Mapping: Spring Data Relational)
10. Draw ERD with ERDPlus (<https://erdplus.com>)
11. Draw ERD with Drawio (<https://app.diagrams.net>)
12. Manage the project with Github (<https://github.com/>)
13. Develop the description with Markdown guide : (<https://www.markdownguide.org/>)
14. Manage and build project with Maven: (<https://maven.apache.org/download.cgi>)
15. Outline the frontend with Figma: (<https://www.figma.com/>)

